

```
from google.colab import files
uploaded=files.upload()
print(uploaded)
```

[Choose Files](#) Housing.csv

- **Housing.csv**(text/csv) - 29981 bytes, last modified: 1/12/2022 - 100% done

Saving Housing.csv to Housing.csv

{'Housing.csv': b'price,area,bedrooms,bathrooms,stories,mainroad,guestroom,basement,hotwaterheating,airconditioning,parki'}

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import math
```

```
house_data=pd.read_csv('Housing.csv')
house_data
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking
0	13300000	7420	4	2	3	yes	no	no	no	yes	
1	12250000	8960	4	4	4	yes	no	no	no	yes	
2	12250000	9960	3	2	2	yes	no	yes	no	no	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	
...	...	...	...	...	...	...	...	...	...	...	...
540	1820000	3000	2	1	1	yes	no	yes	no	no	
541	1767150	2400	3	1	1	no	no	no	no	no	
542	1750000	3620	2	1	1	yes	no	no	no	no	
543	1750000	2910	3	1	1	no	no	no	no	no	
544	1750000	3850	3	1	2	yes	no	no	no	no	

545 rows × 13 columns

Next steps:

[Generate code with house_data](#)

[New interactive sheet](#)

```
house_data.info()
house_data.describe()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   price                 545 non-null   int64
1   area                 545 non-null   int64
2   bedrooms             545 non-null   int64
3   bathrooms            545 non-null   int64
4   stories              545 non-null   int64
5   mainroad            545 non-null   object
6   guestroom           545 non-null   object
7   basement            545 non-null   object
8   hotwaterheating     545 non-null   object
9   airconditioning     545 non-null   object
10  parking              545 non-null   int64
11  prefarea            545 non-null   object
12  furnishingstatus    545 non-null   object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB

```

	price	area	bedrooms	bathrooms	stories	parking
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	545.000000
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	0.693578
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	0.861586
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	0.000000
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	0.000000
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	0.000000
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	1.000000
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	3.000000

```
house_data.head(10)
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parkin
0	13300000	7420	4	2	3	yes	no	no	no	yes	
1	12250000	8960	4	4	4	yes	no	no	no	yes	
2	12250000	9960	3	2	2	yes	no	yes	no	no	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	
5	10850000	7500	3	3	1	yes	no	yes	no	yes	
6	10150000	8580	4	3	4	yes	no	no	no	yes	
7	10150000	16200	5	3	2	yes	no	no	no	no	
8	9870000	8100	4	1	2	yes	yes	yes	no	yes	
9	9800000	5750	3	2	4	yes	yes	no	no	yes	

```
print("Prices accrued on Original Data:"+str(len(house_data.index)))
```

```
Prices accrued on Original Data:545
```

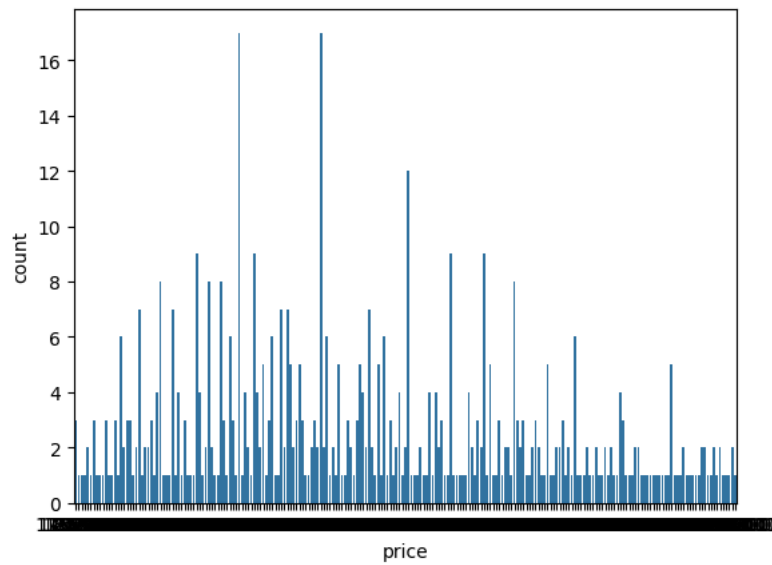
```

#Analysing our data
#sns.countplot(x='mainroad',data=house_data)

```

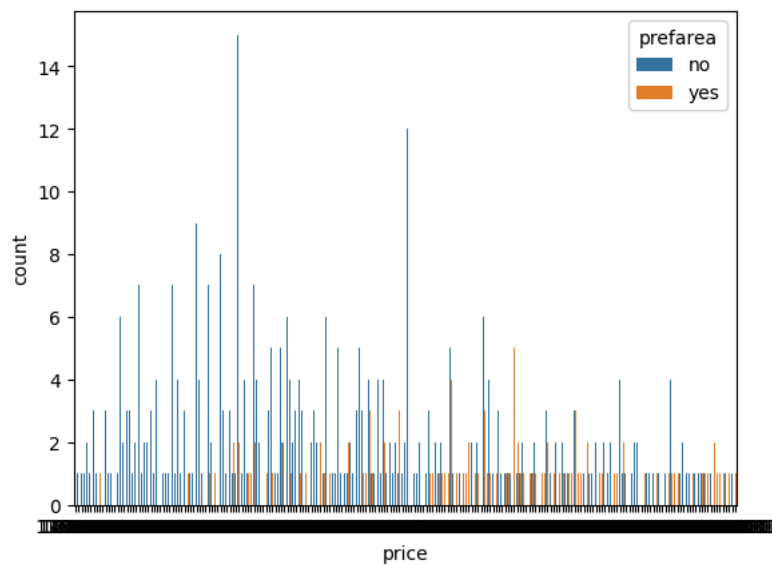
```
sns.countplot(x='price',data=house_data)
```

<Axes: xlabel='price', ylabel='count'>



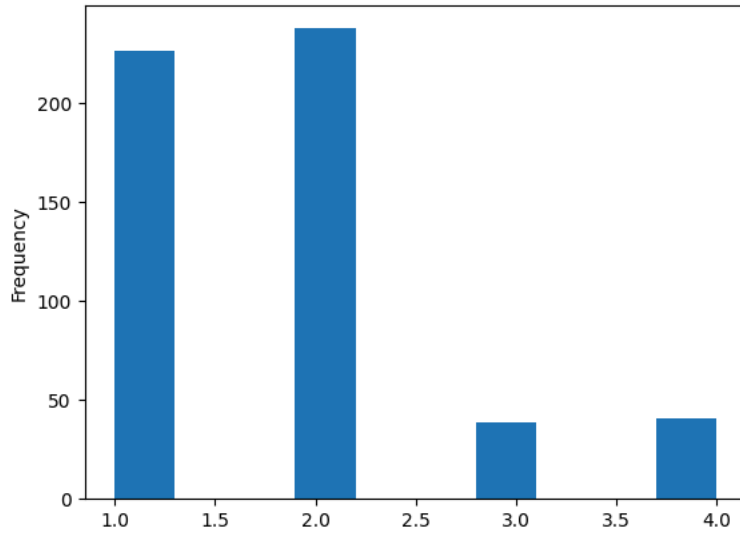
```
sns.countplot(x='price',hue='prefarea',data=house_data)
```

<Axes: xlabel='price', ylabel='count'>



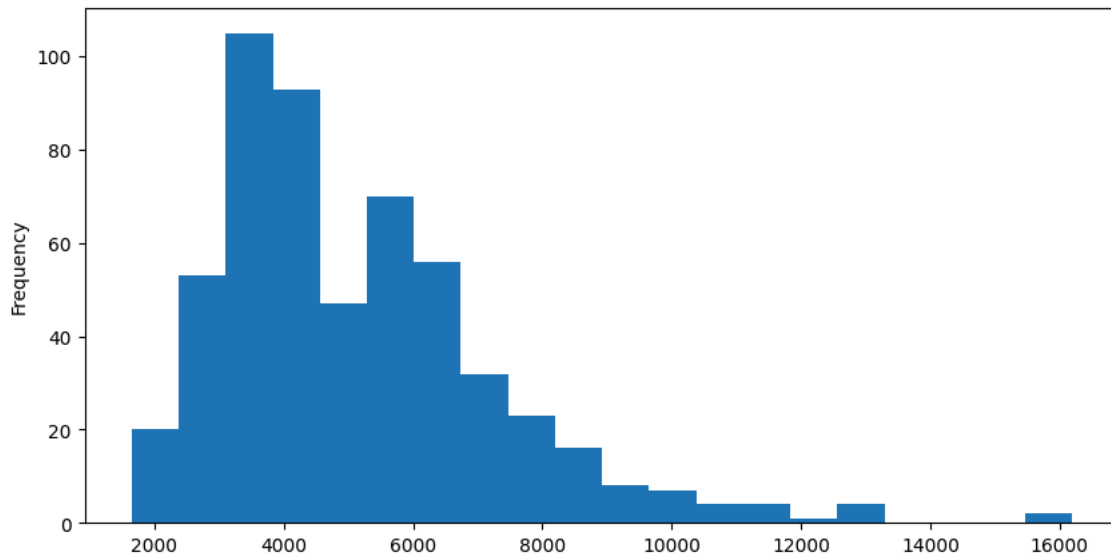
```
house_data['stories'].plot.hist()
```

<Axes: ylabel='Frequency'>



```
house_data['area'].plot.hist(bins=20,figsize=(10,5))
```

<Axes: ylabel='Frequency'>



```
house_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   price               545 non-null   int64
1   area               545 non-null   int64
2   bedrooms           545 non-null   int64
3   bathrooms          545 non-null   int64
4   stories            545 non-null   int64
5   mainroad           545 non-null   object
6   guestroom          545 non-null   object
7   basement           545 non-null   object
8   hotwaterheating    545 non-null   object
9   airconditioning    545 non-null   object
10  parking            545 non-null   int64
11  prefarea           545 non-null   object
12  furnishingstatus   545 non-null   object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
```

```
#Data Wrangling or Data Clean
house_data.isnull()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking
0	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
540	False	False	False	False	False	False	False	False	False	False	False
541	False	False	False	False	False	False	False	False	False	False	False
542	False	False	False	False	False	False	False	False	False	False	False
543	False	False	False	False	False	False	False	False	False	False	False
544	False	False	False	False	False	False	False	False	False	False	False

545 rows × 13 columns

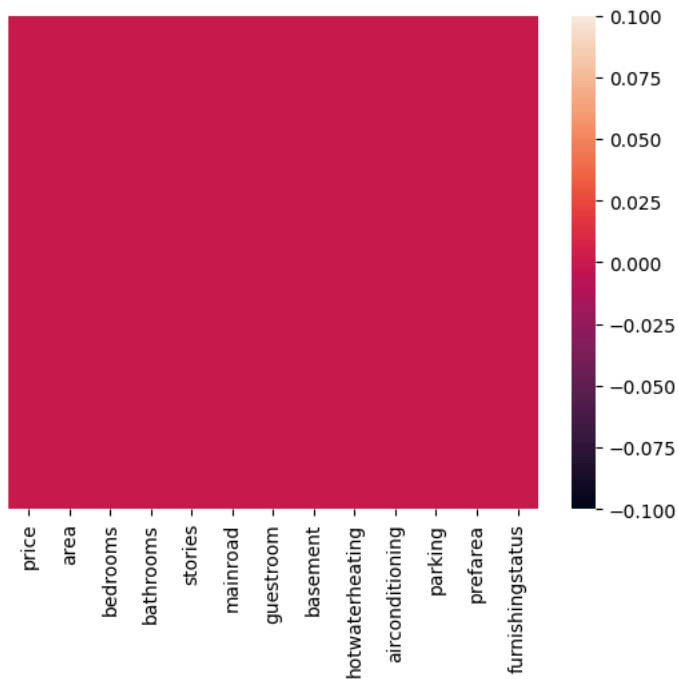
```
house_data.isnull().sum()
```

	0
price	0
area	0
bedrooms	0
bathrooms	0
stories	0
mainroad	0
guestroom	0
basement	0
hotwaterheating	0
airconditioning	0
parking	0
prefarea	0
furnishingstatus	0

dtype: int64

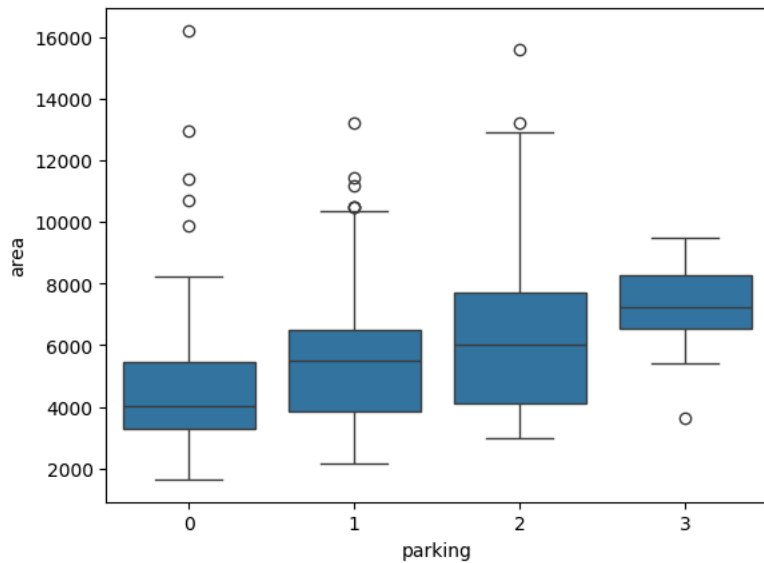
```
sns.heatmap(house_data.isnull(), yticklabels=False)
```

<Axes: >



```
sns.boxplot(x='parking',y='area',data=house_data)
```

<Axes: xlabel='parking', ylabel='area'>



```
house_data.head(5)
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking
0	13300000	7420	4	2	3	yes	no	no	no	yes	2
1	12250000	8960	4	4	4	yes	no	no	no	yes	3
2	12250000	9960	3	2	2	yes	no	yes	no	no	2
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2

```
house_data.drop("bathrooms",axis=1,inplace=True)
house_data.head(5)
```

	price	area	bedrooms	stories	mainroad	guestroom	basement	hotwaterheating	parking	prefarea	furnishingstatus
0	13300000	7420	4	3	yes	no	no	no	2	yes	furnished
1	12250000	8960	4	4	yes	no	no	no	3	no	furnished
2	12250000	9960	3	2	yes	no	yes	no	2	yes	semi-furnished
3	12215000	7500	4	2	yes	no	yes	no	3	yes	furnished
4	11410000	7420	4	2	yes	yes	yes	no	2	no	furnished

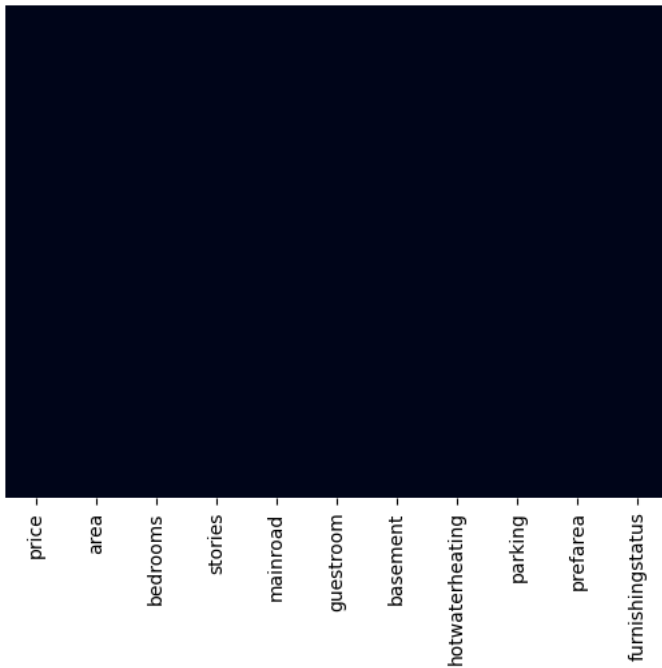
```
sns.heatmap(house_data.isnull(),yticklabels=False,cbar=False)
house_data.isnull().sum()
```

```

0
price      0
area       0
bedrooms   0
stories     0
mainroad   0
guestroom  0
basement   0
hotwaterheating  0
parking    0
prefarea   0
furnishingstatus  0

```

dtype: int64



```
pd.get_dummies(house_data['mainroad'],drop_first=True)
```

```

      yes
0    True
1    True
2    True
3    True
4    True
...    ...
540  True
541  False
542  True
543  False
544  True
545 rows x 1 columns

```

```

embark=pd.get_dummies(house_data['parking'],drop_first=True )
embark.head(5)

```

```

      1      2      3
0  False  True  False
1  False  False  True
2  False  True  False
3  False  False  True
4  False  True  False

```

```

house_data=pd.concat([house_data,embark],axis=1)
house_data.head(5)

```

```

      price  area  bedrooms  stories  mainroad  guestroom  basement  hotwaterheating  parking  prefarea  furnishingstatus
0  13300000  7420         4        3        yes         no         no              no         2         yes         furnished
1  12250000  8960         4        4        yes         no         no              no         3         no         furnished
2  12250000  9960         3        2        yes         no         yes              no         2         yes         semi-furnished
3  12215000  7500         4        2        yes         no         yes              no         3         yes         furnished
4  11410000  7420         4        2        yes         yes        yes              no         2         no         furnished

```

```

#Train and Test Data
x=house_data.drop('parking',axis=1)
y=house_data['parking']

```

```

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=10)
from sklearn.preprocessing import StandardScaler

x_train_unscaled, x_test_unscaled, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=10)

# Apply StandardScaler to the features
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train_unscaled)
x_test = scaler.transform(x_test_unscaled)

```

◆ Gemini

```

from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
lr=LogisticRegression(max_iter=1000) # Increased max_iter for convergence
lr.fit(x_train,y_train)

```



```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to co
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
    LogisticRegression ⓘ ?)
```

```
pred=lr.predict(x_test)
pred
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test,pred)
```

```
array([[83,  0,  0,  0],
       [38,  0,  0,  0],
       [37,  0,  0,  0],
       [ 6,  0,  0,  0]])
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test,pred))
```

```
_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predic
:", len(result))
_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predic
:", len(result))
_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predic
:", len(result))
```

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test,pred)
```

```
0.5060975609756098
```

Train Logistics on Titanic Data set for survival predictions

```
#check Accuracy
from google.colab import files
uploaded=files.upload()
print(uploaded)
```

Choose Files Titanic-Dataset.csv

- **Titanic-Dataset.csv**(text/csv) - 61194 bytes, last modified: 7/12/2026 - 100% done
- Saving Titanic-Dataset.csv to Titanic-Dataset.csv

```
import numpy as np
import matplotlib as plt
import pandas as pd
%matplotlib inline
dataset=pd.read_csv('Titanic-Dataset.csv')
dataset.head(2)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	

Next steps: [Generate code with dataset](#) [New interactive sheet](#)

```
x=dataset.iloc[:,[2,4]].values
y=dataset.iloc[:,4].values
x
y
```

```
female', 'male', 'male', 'male', 'female', 'female', 'male',  
'female', 'male', 'male', 'female', 'male', 'female', 'female',  
'male', 'male'], dtype=object)
```

```
from sklearn.model_selection import train_test_split  
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=10)
```